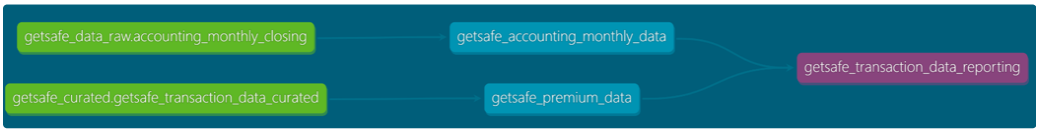
GetSafe Analytics Case Study 1

Flow Chart



Purpose

The purpose of this project is to create a reusable analytics engineering solution to reconcile finance and accounting premium figures. It covers a dbt models that build a finance-facing premium data mart with the columns party, month, and premium from raw transaction data and a robust reporting solution to compare those finance results with accounting's monthly closing table and to generate a systematic discrepancy report. Finally, observing discrepancies between the two sources, along with potential root causes and remediation options.

Data Overview

A csv file is provided which contains raw transaction data.

File name - Data for Analytics Case Study - Raw Transaction Data.csv

getsafe_transaction_data_raw - This table defines the transaction parameters used to create a closing report for the Finance team.

Key Dimensions & Metrics

Core Dimensions

party, month

Engagement Metrics

finance_premium, accounting_premium, variance, discrepancy_outcome, premium_processed, premium_refunded.

Process

The complete process is implemented in DBT and Bigquery for ETL and data modeling steps.

1. The CSV file is uploaded into the seed folder in dbt where we run a dbt seed command to upload the file in BigQuery as raw table - 'getsafe_transaction_data_raw' under 'getsafe_raw' dataset. This raw table serves as the immutable source layer and is not directly consumed for analysis.

2. The raw table is transformed into curated staging models using dbt. This curated table serve as the clean and standardized foundation layer. The following transformations were implemented:
    a. Converted column names to snake_case.
    b. Updated columns to appropriate data types like date, float, string, etc.
    c. Ensured the uniqueness of the important column.
    d. Extracted the processed premium and refunded premium from the data.
    e. Normalized the premium amount.
3. To optimize transformation logic and maintain modularity, intermediate models were created using ephemeral materialization in dbt. This helped the pipeline to avoid unnecessary physical table creation, improve query performance and readability, reduced storage overhead. These ephemeral models are compiled into downstream sql at runtime and are not materialised in Bigquery. Ephemeral model names are ‘getsafe_accounting_monthly_data', ‘getsafe_premium_data’, 'getsafe_transaction_data_enriched.
4. The final transaction table is created for the engineers to compare the numbers between accounting and finance premium with some extra columns like ‘variance', ‘variance_%’, 'discrepancy_outcome’ for further analysis and to make desicion.
5. The hypothesis tables are created to provide better understanding of the discrepancies in the data. These tables presents the change in data when a certain hypothetical requirement is fulfilled.

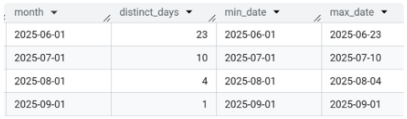
Tables Created

Layer	Dataset	Table
Raw layer	`getsafe-bigquery-prod.getsafe_data_raw`	`accounting_monthly_closing` , `getsafe_transaction_data_raw`
Curated layer	`getsafe-bigquery-prod.getsafe_curated`	`getsafe_transaction_data_curated`
Reporting Layer	`getsafe-bigquery-`	`getsafe_transaction_data_reporting`

	`prod.getsafe_reporting`	
Hypothesis	`getsafe-bigquery-prod.getsafe_case_study_hypothesis`	`getsafe_hypothesis_1` `getsafe_hypothesis_2` `getsafe_hypothesis_3`

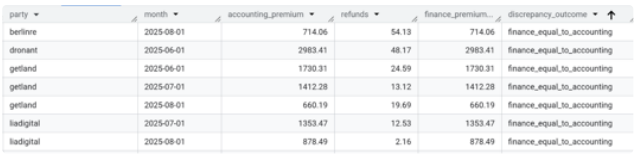
Discrepancies Reason Hypothesis

1. The transaction dates available in the CSV are incomplete. For example, for the June–August period there are missing dates, such as only some dates like in June(23), July(10), and August(4) are present, which suggests that the file does not fully cover the entire reporting period.



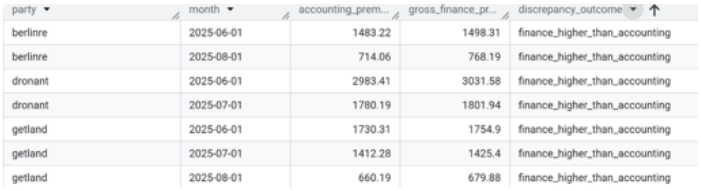
month	distinct_days	min_date	max_date
2025-06-01	23	2025-06-01	2025-06-23
2025-07-01	10	2025-07-01	2025-07-10
2025-08-01	4	2025-08-01	2025-08-04
2025-09-01	1	2025-09-01	2025-09-01

2. There is ambiguity around how transaction status is interpreted. It is unclear whether both process and processed should be treated as “processed” from an accounting perspective, and therefore whether the normalized premium amount in Finance should include transactions with both statuses or only one of them. (This hypothesis resulted in finance and accounting premium to be equal in most of the cases which means that the accounting is only canculating the processed premiums and not the net premium.)



party	month	accounting_premium	refunds	finance_premium...	discrepancy_outcome
berlinre	2025-08-01	714.06	54.13	714.06	finance_equal_to_accounting
dronant	2025-06-01	2983.41	48.17	2983.41	finance_equal_to_accounting
getland	2025-06-01	1730.31	24.59	1730.31	finance_equal_to_accounting
getland	2025-07-01	1412.28	13.12	1412.28	finance_equal_to_accounting
getland	2025-08-01	660.19	19.69	660.19	finance_equal_to_accounting
liadigital	2025-07-01	1353.47	12.53	1353.47	finance_equal_to_accounting
liadigital	2025-08-01	878.49	2.16	878.49	finance_equal_to_accounting

3. Accounting and Finance may be using different premium definitions. One possibility is that the accounting team calculates gross premium (including process, processed, and refunded amounts), whereas the finance model currently derives net premium by excluding or reversing refunded transactions. (Seems like the finance premium is higher than accounting which means that this hypothesis can be ruled out.)



party	month	accounting_prem...	gross_finance_pr...	discrepancy_outcome
berlinre	2025-06-01	1483.22	1498.31	finance_higher_than_accounting
berlinre	2025-08-01	714.06	768.19	finance_higher_than_accounting
dronant	2025-06-01	2983.41	3031.58	finance_higher_than_accounting
dronant	2025-07-01	1780.19	1801.94	finance_higher_than_accounting
getland	2025-06-01	1730.31	1754.9	finance_higher_than_accounting
getland	2025-07-01	1412.28	1425.4	finance_higher_than_accounting
getland	2025-08-01	660.19	679.88	finance_higher_than_accounting

4. The raw CSV contains duplicate transaction IDs. These duplicates were de-duplicated when building the reporting layer, but there is a high likelihood that similar duplicate rows were counted manually by the accounting team, which would inflate the accounting totals relative to the finance-derived figures.
5. Timing of status changes may differ between systems. Transaction statuses in the operational data can be updated after the accounting team has already completed their manual monthly close, which means the later Finance view can include adjustments that were not reflected in the original accounting numbers.
6. The month definition may not be aligned between datasets. The Month column in the accounting table may be derived from a different date field (for example, invoice date or aquisition date) or from a non-standard period definition (such as a month running from the first Monday instead of the calendar 1st to end-of-month), leading to mismatches when compared to Finance's calendar-month aggregation.
7. Currency treatment may differ between Accounting and Finance. The finance model appears to include only EUR-denominated transactions, whereas the accounting figures may consolidate multiple currencies into a single amount (for example, EUR plus other currencies converted at internal rates), which would naturally create differences between the two totals.

Challenges

1. Partitioning of data by a date column can help reduce table scan volume. As I am using a Bigquery non billable account the partitions are automatically reduced to last 60 days which was creating a data upload discrepencies as the dates provided in the raw data are from last year or are older than the last 60 days. (But I have mentioned the partition_by clause in the code before commiting so you will be able to replicate at your end.)
2. Accounting data is unclear as only the party, month, and premium columns are given which created challenges to confirm my hypothesis.

Further Data Improvements

1. Create a DAG or a scheduler for daily refresh.
2. Create incremental dbt model to reduce the processing cost.
3. Store large csv files in the google cloud bucket to maintain the raw data.

Repository Details
You can find the complete code structure and logic here -

GitHub - PreetikaJ/getsafe_analytics_dbt_mart: Analytics Engineering case study for Getsafe, implementing scalable dbt models for premium reconciliation and KPI data marts. Covers financial data modeling, discrepancy analysis, and customer lifecycle metrics optimized for BI consumption.